

Recursion and Backtracking

What is Recursion

Recursion is when a function calls itself to solve smaller instances of the same problem, until it reaches a 'base case' that stops the recursion. Every recursive function needs: a base case (stopping condition) and a recursive case (progress toward the base case).

Anatomy of a Recursive Function

```
def factorial(n):  
  
    if n <= 1:          # base case  
  
        return 1  
  
    return n * factorial(n - 1)  # recursive case  
  
# Call stack for factorial(4):  
# factorial(4) -> 4 * factorial(3)  
#               -> 4 * (3 * factorial(2))  
#               -> 4 * (3 * (2 * factorial(1)))  
#               -> 4 * (3 * (2 * 1)) = 24
```

The Call Stack

Each recursive call is pushed onto the program's call stack and stays there until it returns. Too many recursive calls without a reachable base case causes a Stack Overflow error. This is directly connected to the Stack data structure covered earlier.

What is Backtracking

Backtracking is a refinement of recursion used to solve problems by building a solution incrementally, and abandoning ('backtracking' from) a path as soon as it's determined that path cannot lead to a valid solution. It's essentially DFS through a decision tree of choices.

Backtracking Template

```
def backtrack(path, choices):  
  
    if is_solution(path):  
  
        record(path)  
  
    return
```

```
for choice in choices:
    if is_valid(choice, path):
        path.append(choice)          # make the choice
        backtrack(path, choices)    # explore
        path.pop()                  # undo the choice (backtrack)
```

Classic Backtracking Problems

- N-Queens — place N queens on a board so none attack each other.
- Generate all permutations or combinations of a list.
- Subsets — generate the power set of a list.
- Sudoku Solver.
- Word Search in a grid.
- Combination Sum — find combinations that sum to a target.

Recursion vs Iteration

- Recursion is often more readable for tree/graph problems and divide-and-conquer algorithms.
- Iteration is usually more memory-efficient (no call stack overhead) and preferred when performance is critical.
- Any recursive solution can be rewritten iteratively using an explicit stack — worth knowing for interviews that ask you to avoid recursion.